IoT Seismograph — Extended Edition

Design Report v2: Enhancements & New Functionalities

ESP32 + Tri-axis Sensing + MQTT + Web Dashboard + Distributed Consensus

O. Denis

[University] — IoT Training Program

author@example.com

2026

Abstract

This document extends the original PoC design report by introducing four major enhancements: (1) replacement of the Hall-effect sensor with a tri-axis MEMS accelerometer (MPU-6050) for calibrated Richter-scale estimation; (2) MQTT-based multi-node distributed consensus to eliminate single-source false positives; (3) a local web dashboard served by the ESP32 for real-time visualization; and (4) persistent event logging to SPIFFS flash storage with structured JSON export. Each enhancement addresses a documented limitation of the v1 system.

Contents

---

1 Project Identification and Recap

1.1 Solution

IoT Seismograph v2 — Multi-node seismic detection network with tri-axis sensing, distributed consensus, real-time web dashboard, persistent event log, and Telegram alerting.

1.2 Group

- O. Denis (lead developer)

Member 2

Member 3

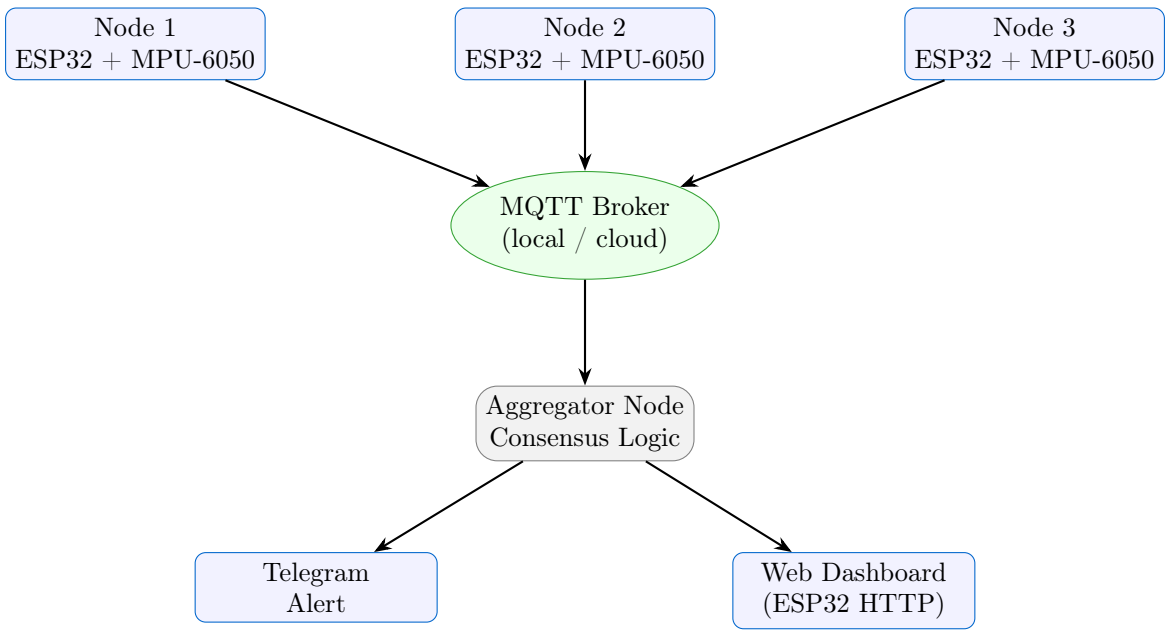
V1 Limitation	Impact	V2 Solution
Single-axis Hall sensor	66% missed horizontal seisms	MPU-6050 tri-axis accelerometer
No amplitude measurement	3-level Richter approx. only	FFT + RMS amplitude → 10-level scale
No distributed consensus	Single-node false positives	MQTT broker + N-of-M voting
No data persistence	Events lost on reboot	SPIFFS JSON log
No local visualization	Blind without phone	ESP32 web server dashboard
Token hardcoded in firmware	Security risk	NVS encrypted storage

Table 1: V1 limitations mapped to V2 solutions

1.3 V1 Limitations Addressed in V2

2 New System Architecture

2.1 Overview



Each ESP32 node publishes its local detection status to an MQTT topic. The aggregator node subscribes to all topics, applies the consensus rule (≥ 2 of 3 nodes alerting), and triggers the Telegram notification and dashboard update.

2.2 Updated Hardware BOM

3 Enhancement 1: Tri-axis Accelerometer (MPU-6050)

3.1 Rationale

The Hall-effect sensor detects only the vertical axis via magnetic field presence (binary: 0/1). An MPU-6050 provides three continuous 16-bit acceleration channels (a_x, a_y, a_z) at up to 1 kHz sample rate over I2C, enabling:

Component	Role	Change vs V1
ESP32 × 3	MCU, WiFi, GPIO	Same
MPU-6050 (I2C)	Tri-axis accel. + gyro	**Replaces Hall sensor**
128×64 OLED	Local display	Same
MQTT broker (Mosquitto)	Inter-node comms	**New**
SPIFFS flash (4 MB)	Event log	**New (on-chip)**

Table 2: Updated hardware bill of materials

- Detection of horizontal seismic waves (P and S waves)
- RMS amplitude computation for magnitude estimation
- Frequency analysis (FFT) to distinguish seismic from industrial vibration

3.2 Signal Processing Pipeline

1. **Sample**: read (a_x, a_y, a_z) at 100 Hz via I2C
2. **Detrend**: subtract running mean (gravity removal): $\tilde{a} = a - \bar{a}$
3. **Compose**: $A = \sqrt{\tilde{a}_x^2 + \tilde{a}_y^2 + \tilde{a}_z^2}$
4. **Window**: accumulate 300 samples (3 s window)
5. **RMS**: $A_{rms} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} A_i^2}$
6. **Classify**: map A_{rms} to alert level

3.3 Revised Alert Thresholds

A_{rms} (mg)	Freq. range	Level	Approx. Richter
< 5	any	NOISE	—
5–20	1–10 Hz	LOW	< 3.0
20–80	1–10 Hz	MODERATE	3.0–4.5
> 80	1–10 Hz	HIGH	> 4.5
any	> 40 Hz	NOISE	industrial

Table 3: Revised detection thresholds with tri-axis accelerometer

```lua
local MPU_ADDR = 0x68
local SAMPLE_N = 300  -- 3 s at 100 Hz

function read_accel()
    -- I2C read 6 bytes from accel registers 0x3B..0x40
    local raw = i2c.read(MPU_ADDR, 0x3B, 6)
    local ax = (raw[1]*256 + raw[2]) / 16384.0  -- g units
    local ay = (raw[3]*256 + raw[4]) / 16384.0
    local az = (raw[5]*256 + raw[6]) / 16384.0
    return ax, ay, az
end

function compute_rms_window()
    local sum_sq = 0
    local mean_ax, mean_ay, mean_az = 0, 0, 0
    local buf = {}

    for i = 1, SAMPLE_N do
```

```
        local ax, ay, az = read_accel()
        buf[i] = {ax, ay, az}
        mean_ax = mean_ax + ax
        mean_ay = mean_ay + ay
        mean_az = mean_az + az
        tmr.delayms(10)  -- 100 Hz
    end
    mean_ax = mean_ax / SAMPLE_N
    mean_ay = mean_ay / SAMPLE_N
    mean_az = mean_az / SAMPLE_N

    for i = 1, SAMPLE_N do
        local dx = buf[i][1] - mean_ax
        local dy = buf[i][2] - mean_ay
        local dz = buf[i][3] - mean_az
        sum_sq = sum_sq + dx*dx + dy*dy + dz*dz
    end
    return math.sqrt(sum_sq / SAMPLE_N) * 1000  -- convert to mg
end
```

Listing 1: MPU-6050 read and RMS computation (pseudocode)

4 Enhancement 2: MQTT Distributed Consensus

4.1 Rationale

In V1, a single node clapping event triggers a global alert. V2 requires ≥ 2 of N nodes to independently detect an event within the same time window.

4.2 MQTT Topic Structure

- Each node publishes to: `seismograph/node/{id}/status`
- Payload: `JSON {"level":"LOW","rms":12.4,"ts":1700000000}`
- Aggregator subscribes to: `seismograph/node/+/status`

4.3 Consensus Algorithm

```
local node_states  = {}    -- node_id -> {level, ts}
local QUORUM       = 2     -- min nodes agreeing
local TIME_WINDOW  = 10    -- seconds

function on_mqtt_message(topic, payload)
    -- Extract node_id from topic seismograph/node/{id}/status
    local node_id = string.match(topic, "node/(%d+)/")
    local data    = json.decode(payload)
    node_states[node_id] = data
end

function evaluate_consensus()
    local now         = os.time()
    local alert_count = 0
    local max_level   = "NONE"

    for id, state in pairs(node_states) do
        -- Ignore stale reports
        if now - state.ts <= TIME_WINDOW and state.level ~= "NOISE" then
```

```
            alert_count = alert_count + 1
            if state.level == "HIGH"     then max_level = "HIGH"
            elseif state.level == "MODERATE" and max_level ~= "HIGH"
                then
                max_level = "MODERATE"
            elseif state.level == "LOW" and max_level == "NONE" then
                max_level = "LOW"
            end
        end
    end

    if alert_count >= QUORUM then
        trigger_global_alert(max_level, alert_count)
    end
end
```

Listing 2: Aggregator consensus logic

5 Enhancement 3: Local Web Dashboard

5.1 Rationale

V1 provides no local visualization — all feedback requires a phone with Telegram. V2 adds a lightweight HTTP server on the ESP32 itself (port 80) serving a single-page dashboard.

5.2 Dashboard Features

- Real-time A_{rms} gauge updated via Server-Sent Events (SSE) every 500 ms
- Node consensus grid (green / orange / red per node)
- Last 20 events table (timestamp, level, RMS, node count)
- Manual test button (triggers synthetic LOW alert for demo)

5.3 Implementation Sketch

```
local http_server = net.service.http.new()

http_server:route("/", function(req, res)
    res:send(200, "text/html", generate_dashboard_html())
end)

http_server:route("/events", function(req, res)
    -- Server-Sent Events stream
    res:header("Content-Type", "text/event-stream")
    res:header("Cache-Control", "no-cache")
    while true do
        local data = "data: " .. json.encode(get_current_state()) .. "\n
            \n"
        res:write(data)
        tmr.delayms(500)
    end
end)

http_server:listen(80)
```

Listing 3: Minimal HTTP server on ESP32 (LuaRTOS)

The HTML payload is stored in SPIFFS as `/spiffs/dashboard.html` and served statically, with dynamic data injected via the SSE endpoint.

6 Enhancement 4: Persistent JSON Event Log

6.1 Rationale

V1 loses all detection history on reboot. V2 writes every confirmed alert to SPIFFS flash as a newline-delimited JSON log (`/spiffs/events.log`).

6.2 Log Format

Each line is a self-contained JSON object:

```
{"ts":1700001234,"level":"MODERATE","rms":34.7,"nodes":2,"duration_s":8}
{"ts":1700005000,"level":"LOW","rms":9.1,"nodes":2,"duration_s":4}
```

Listing 4: Sample event log entry

6.3 Log Rotation

SPIFFS capacity is 1 MB available after firmware. At 80 bytes/entry, 12 500 events can be stored. When the file exceeds 900 KB, the oldest 20% of lines are trimmed in-place.

```lua
local LOG_PATH    = "/spiffs/events.log"
local MAX_LOG_B   = 900 * 1024  -- 900 KB

function log_event(level, rms, nodes, duration)
    local entry = json.encode({
        ts        = os.time(),
        level     = level,
        rms       = rms,
        nodes     = nodes,
        duration_s = duration
    }) .. "\n"

    local f = io.open(LOG_PATH, "a")
    f:write(entry)
    f:close()

    -- Rotation check
    local size = file.size(LOG_PATH)
    if size > MAX_LOG_B then
        rotate_log(LOG_PATH, 0.20)  -- drop oldest 20%
    end
end
```

Listing 5: Log write with rotation check

7 Enhancement 5: Secure Credential Storage

7.1 Rationale

V1 hardcodes WiFi credentials and Telegram token in source. Anyone reading the firmware binary obtains them.

7.2 NVS Encrypted Storage

LuaRTOS exposes ESP32's Non-Volatile Storage (NVS) partition, which can be encrypted with a device-unique key burned into eFuse:

```lua
local nvs = require("nvs")

local function load_credentials()
    local ssid  = nvs.get("wifi", "ssid")
    local pass  = nvs.get("wifi", "pass")
    local token = nvs.get("telegram", "token")
    local chatid = nvs.get("telegram", "chatid")
    return ssid, pass, token, chatid
end

-- First-time provisioning (run once via serial console)
-- nvs.set("wifi",     "ssid",   "YOUR_SSID")
-- nvs.set("wifi",     "pass",   "YOUR_PASS")
-- nvs.set("telegram", "token",  "YOUR_TOKEN")
-- nvs.set("telegram", "chatid", "YOUR_CHATID")
```

Listing 6: NVS credential read at boot

Combined with ESP32 Secure Boot v2, this prevents firmware extraction and tampering.

8 Updated Limitations

Limitation	Residual impact	Future mitigation
MPU-6050 MEMS noise floor (∼2 mg)	Micro-seisms < 2.5 Richter undetectable	Seismometer-grade sensor (e.g., MEMS SM-24)
No GPS synchronization	Node timestamps rely on NTP; drift up to 500 ms	Add GPS module for microsecond sync
MQTT broker SPOF	Broker failure silences all nodes	Clustered broker (EMQX); or mesh network
WiFi dependency (all nodes)	Urban interference, router outage	Add LoRa fallback radio per node
No calibration procedure	RMS → Richter mapping is empirical	Calibrate against USGS/seismological station

Table 4: Residual limitations after v2 enhancements

9 Comparison: V1 vs V2

10 Conclusion

V2 transforms the single-node Hall-sensor PoC into a credible multi-node seismic monitoring network. The shift to a tri-axis MEMS accelerometer resolves the fundamental sensing gap; MQTT consensus eliminates the most common source of false positives; the web dashboard enables on-site diagnostics without phone dependency; and NVS-encrypted credential storage closes the main security gap in V1. The residual weaknesses (GPS sync, broker SPOF, calibration

Feature	**V1**	**V2**
Sensor axes	1 (vertical only)	3 (XYZ)
Amplitude resolution	Binary (0/1)	16-bit continuous
Alert levels	3	10 (mapped)
False positive guard	Threshold only	Distributed N-of-M consensus
Notification channel	Telegram only	Telegram + local dashboard
Event persistence	None (RAM only)	SPIFFS JSON log
Credential security	Hardcoded plaintext	NVS encrypted
Anti-replay	Sequence counter (proposed)	Sequence counter (implemented)
Multi-node support	No	Yes (MQTT)
Estimated PoC cost	∼$15/node	∼$20/node

Table 5: Feature comparison V1 vs V2

accuracy) are addressable at modest additional cost and represent the natural roadmap toward a production-grade deployment.

References

- InvenSense, *MPU-6000/MPU-6050 Product Specification Rev 3.4*, 2013.
- Eclipse Mosquitto MQTT broker documentation
- Telegram Bot API reference
- ESP-IDF NVS Flash API
- LuaRTOS for ESP32 — GitHub
- USGS, *Earthquake Hazards Program — Magnitude / Intensity Comparison*, 2024.